Consteval-only Values for C++26

Document #:	P4101R0 [Latest] [Status]
Date:	2026-03-24
Project:	Programming Language C++
Audience:	EWG, CWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com> Peter Dimov <pdimov@gmail.com> Daveed Vandevoorde <dvandevoorde@nvidia.com> Dan Katz <katzdm@gmail.com>

Contents

§ 1 Introduction

The Reflection design from [P2996R13] was based on a model of having consteval-only types to prevent reflections from leaking to runtime. But we've run into issues and limitations with that approach, so we propose that, for C++26, we change instead to a consteval-only value model. This solves the same problems, but has additional benefits.

§ 2 Issues with Consteval-only Types

The consteval-only type model is simple. Any type with `std::meta::info` in it somewhere is consteval-only. This includes chasing pointers and references, function types, class members, and pointer-to-member types. There has been some desire voiced to simply allow `info` to persist to runtime, and having it simply be an empty type in which nothing is observable, but up until recently, there weren't really any problems expressed with the model itself.

However, Jakub Jelinek pointed out in January 2026 that whether a type is consteval-only is not actually a static property. This led to [CWG3150] (Incomplete consteval-only class types):

> A class type is consteval-only depending on its member types. However, a class type may be incomplete, and thus the question cannot be answered where needed.
>
> For example,
>
> ```
> struct S;
> void f(S*);   // #1
> struct S {    // #2
>   std::meta::info x;
> };
> ```
>
> Does the class definition at #2 make the function declaration #1 retroactively ill-formed? What if #1 and #2 are not mutually reachable?

This is a problem. Note also that class incompleteness might apply to members — rather than `S` being incomplete, `S` could have a member `U*` where `U` is incomplete. We cannot say definitively whether a type is consteval-only or not.

Discussing this issue led to the discovery of a closely-related problem:

```
template <class T> struct S;
void f(S<int>*);
```

Consteval-only types include function types. Does the above need to instantiate `S` to determine whether `S` is consteval-only, because the completeness of `S<int>` affects the semantics of the program? Currently, we do *not* need to instantiate `S` there, and indeed lots of code relies on this lack of instantiation.

§ 2.1 Consteval-only Types are Abstract Types

One suggested approach of how to resolve these issues (the incompleteness issue and the extra-instantiation issue) was to borrow from the closest analogue we have to this problem in C++ today: abstract class types. [P0929R2] (Checking for abstract class types) opened with:

> Subclause 13.4 [class.abstract] paragraph 3 states:

> An abstract class shall not be used as a parameter type, as a function return type, or as the type of an explicit conversion. Pointers and references to an abstract class can be declared.

This is troublesome, because it requires checking a property only known once a type is complete at a position (declaring a function) where the rest of the language permits incomplete types to appear.

In practice, this introduces spooky "backward-in-time" errors:

```
struct S;
S f();   // #1, ok

// lots of code

struct S { virtual void f() = 0; };  // makes #1 retroactively ill-
        formed
```

Which is pretty familiar!

That paper refined the ways in which we handle abstract class type checking by only checking for abstractness at the point of definition. The declaration of `f()` above there is fine, but the definition of `f()` would require (and check) that `S` is not abstract.

Note, however, that pointers and references to abstract class types are not themselves abstract class types. This led to the question…

§ 2.2 Must Pointers and References to Consteval-only Types be Consteval-only?

Consider the following example:

```
constexpr std::meta::info r = ^^int;

/* not constexpr */ std::meta::info const* p = &r;
constexpr std::meta::info const* q = &r;
```

`r` does not exist at runtime, so the declaration of `p` needs to be ill-formed, while the declaration of `q` is fine. If `std::meta::info const*` is a consteval-only type, then the declaration of `p` is rejected by our consteval-only type rule in 6.9.1 [basic.types.general]/12:

> 12 […] Every object of consteval-only type shall be
>
> (12.8) — the object associated with a constexpr variable or a subobject thereof,
>
> (12.9) — a template parameter object ([temp.param]) or a subobject thereof, or

(12.10) — an object whose lifetime begins and ends during the evaluation of a core constant expression.

The declaration of `q` is fine.

But if `std::meta::info const*` is no longer a consteval-only type, what happens? The reasoning gets surprisingly complicated.

For `p`, `&r` is an immediate-escalating expression (because `r` has consteval-only type), which means it has to be in an immediate function context, which means it has to be manifestly constant-evaluated, which means that the full-initialization has to be a constant expression. But now we violate what used to be called the "permitted result of a constant expression" rule in 7.7 [expr.const]/21:

21 A *constant expression* is either

(21.1) — a glvalue core constant expression […] or

(21.2) — a prvalue core constant expression whose result object ([basic.lval]) satisfies the following constraints:

(21.2.1) — […]

(21.2.5) — unless the value is of consteval-only type,

(21.2.5.1) — no constituent value of pointer-to-member type points to a direct member of a consteval-only class type,

(21.2.5.2) — no constituent value of pointer type points to or past an object whose complete object is of consteval-only type, and

(21.2.5.3) — no constituent reference refers to an object whose complete object is of consteval-only type.

Status quo, we don't go into 21.2.5 because `std::meta::info const*` is a consteval-only type. But if it weren't, we do, and we violate 21.2.5.2 because `&r` points to an object whose complete type is consteval-only. Which, great, `p` is rejected.

However, `q` is *also* rejected for the exact same reason. The declaration of `q` is also ill-formed. And this is a real problem because:

```
constexpr std::meta::info arr[] = {^^int, ^^char};
constexpr std::span<std::meta::info const> s = arr;
```

Status quo: this is fine. And it is very important that this work because this is our workaround for the lack of non-transient constexpr allocation via `std::define_static_array()`, so approximately everybody who uses reflection will run into needing this.

But if `std::meta::info const*` is no longer a consteval-only type, then `std::span<std::meta::info const>` wouldn't be either, and again we violate the "permitted result" rule because we have a non-conste-

val-only type with a constituent value of pointer type that points to an object of consteval-only type.

§ 2.3 What is the Conclusion?

The conclusion here is that in the consteval-only type model, we simply have to follow all the type edges. Pointers, references, functions, class subobjects, etc. But once we do that, we end up with a significantly more complex rule for handling incomplete types — see [P4132R0] (Addressing incomplete consteval-only types) and [P4135R0] (Consteval-only Types: A design space overview). Is that rule carrying its weight?

Note for interest that the Zig programming language has consteval-only types in the exact way that we want to define here (comptime-only in their parlance), but they don't have incomplete types in the same way, so they don't run into this issue.

§ 3 Consteval-only Values

There is a different approach to restricting certain values to not persist until runtime: enforce the rules at a *value* level instead of a *type* level. It's not objects of type `std::meta::info` that must live at compile-time, it's that reflection values must live at compile-time.

C++ today already has a notion of a value that is only allowed to exist during compile time: consteval functions. We are already used to the fact that our constant rules are value-based rather than type based:

```
consteval auto add(int x, int y) -> int { return x + y; }
constexpr auto sub(int x, int y) -> int { return x - y; }

constexpr auto p = add; // error
constexpr auto q = sub; // ok
```

`add` and `sub` have the same type, as would `p` and `q`, but one of those declarations is valid while the other is not.

And we also have a notion of consteval-only value to specifically close the holes left by the consteval-only type model:

```
constexpr std::meta::info r = ^^int;
void const* p = (void const*)&r; // error
```

C++26 already rejects the declaration of `p` (as it must), but it cannot do so on the basis of types — because `void const*` is of course not consteval-only. Instead, we already do it today on the basis of the expression `(void const*)&r` having consteval-only value. From the rule cited earlier

> 21 A *constant expression* is either
>
> (21.1) — a glvalue core constant expression [...] or

(21.2) — a prvalue core constant expression whose result object ([basic.lval]) satisfies the following constraints:

(21.2.1) — […]

(21.2.5) — unless the value is of consteval-only type,

(21.2.5.1) — no constituent value of pointer-to-member type points to a direct member of a consteval-only class type,

(21.2.5.2) — no constituent value of pointer type points to or past an object whose complete object is of consteval-only type, and

(21.2.5.3) — no constituent reference refers to an object whose complete object is of consteval-only type.

The rules we have today escalate `&r` because it has consteval-only type, but rejects `(void const*)&r` because it has consteval-only value. Just not in those specific words. In other words, the consteval-only type model is already incomplete unless it accounts for consteval-only values.

We already explored this idea of consteval-only values in [P3603R1] (Consteval-only Values and Consteval Variables), but what if we generalized the notion we have today and combined it with reflections? Informally:

> A value is *consteval-only* if it is either
>
> 1. a reflection value or
> 2. a pointer or pointer-to-member that points to either an immediate object or an immediate function.
>
> An object is an *immediate object* if its complete object has either
>
> 1. a constituent value that is consteval-only or
> 2. a constituent reference that refers to either an immediate object or an immediate function.
>
> A variable is an *immediate variable* if the object it declares is an immediate object.

And then we split our term "constant expression" into two: we say a constant expression is allowed to have consteval-only value (it cannot right now) and add a refinement called an *immediate constant expression* which does have consteval-only value. We then say that a `constexpr` variable can be initialized with an immediate constant expression (this escalates it to an immediate variable) but a regular variable cannot be (because otherwise our consteval-only value would not be very consteval-only).

Going back to our earlier example:

```
constexpr std::meta::info r = ^^int;
```

```
/* not constexpr */ std::meta::info const* p = &r;
constexpr std::meta::info const* q = &r;
```

The new way of thinking about these declarations is:

— For `r`: `^^int` is a consteval-only value (it is a reflection), so this is an immediate constant expression. `r` is declared `constexpr`, so it is allowed to have consteval-only value. It becomes an immediate variable.

— For `p`: `&r` is a consteval-only value (it is a pointer to an immediate variable), so this is an immediate constant expression. `p` is not declared `constexpr`, so its initializer is not allowed to have consteval-only value, so this is ill-formed.

— For `q`: As above, `&r` is a consteval-only value, but because `q` is declared `constexpr`, it is allowed to be initialized with an immediate constant expression. It becomes an immediate variable.

Similar reasoning shows that in this other example, both `arr` and `s` are valid declarations and become immediate variables.

```
constexpr std::meta::info arr[] = {^^int, ^^char};
constexpr std::span<std::meta::info const> s = arr;
```

§ 3.1 As a Solution to CWG 3150

Because values (unlike types) during constant evaluation are always complete, we mostly avoid any issues arising from incomplete types. This side-steps the issue from [CWG3150].

There are still some cases that would become IFNDR though:

```
// TU #1
struct S;
extern S const s;
S const* p = &s;

// TU #2
struct S { std::meta::info r; };
constexpr S s = {.r = ^^int};
```

In TU #2, `s` is an immediate variable. It will not persist to runtime. Attempting to use its address in TU #1 just won't link, because there won't be a definition. But that's okay.

But the value approach also means that we don't even have to think about the instantiation issue:

```
template <class T> struct S;
void f(S<int>*);
```

If `S<int>` has a member of type `std::meta::info` and that member is initialized with `^^int`, then that object will necessarily have to live at compile-time, and you just won't be able to invoke a non-`constexpr` function with a pointer to it. It might be a bug to have not declared it `constexpr`, but that simply makes it useless — not problematic.

§ 3.2 The Null Reflection Value

One question that comes up with this approach is what we do about the null reflection value. Currently, that is considered a reflection value:

> [17] A value of type `std::meta::info` is called a *reflection*. There exists a unique *null reflection*; every other reflection is a representation of […]

Is the null reflection value equivalent to a null pointer (i.e. not a consteval-only value) or is it a reflection value (i.e. consteval-only)? If the null reflection value is not consteval-only, then you can have a runtime value of type `std::meta::info` — as long as it is null.

This choice affects how we formulate the rules for immediate-escalation. The following examples assume a:

```
struct S { std::meta::info r; };
```

Null is consteval-only	Null is not consteval-only

We have to be able to reject all of
`std::meta::info a;`
`S b;`
`auto normal() -> void {`
`  std::meta::info c;`
`  S d;`
`}`
Unlike other reflection values, these are not produced during constant-evaluation, so this would have to be a type-based check.
	We have to *allow* all the code at the left, but also be able to disallow the following:
	`std::meta::info a;`
	`constexpr std::meta::info r = ^^int;`
	`auto normal() -> void {`
	`  a = std::meta::info(); // ok`
	`  a = ^^int; // error`
	`  a = r; // error`
	`  a == a; // ok, actually`
	`  a == ^^int; // error`
	`  a == r; // error`
	`  new (&a) std::meta::info(); // ok`
	`  new (&a) std::meta::info(^^int); // error`
	`  S{}; // ok`

```
    S{.r={}}; // ok
    S{.r=^^int}; // error
}
```

In this case, all the reflection values that we need to be consteval-only are produced by during constant evaluation — so we can hook entirely in there.

If the null reflection is consteval-only, we have to immediate-escalate *any* initialization of a `std::meta::info` object. This leads to the interesting consequence that reflections being consteval-only *implies* the existence of a consteval-only type, but a different form than what is in the standard today. This form does not follow pointer, reference, function, or even union edges. But you probably need type-checking in order to reject all the initializations you need to.

If the null reflection is *not* consteval-only, we have to immediate-escalate any creation of a reflection (e.g. a *`reflect-expression`*) — which is analogous in treatment to how we handle immediate functions today. Indeed, a null reflection value is to a reflection value what a null pointer is to a pointer to an immediate function. The analogous example for immediate functions would be:

```
consteval auto f() -> void { }
auto g() -> void { }

void (*p)();
auto normal_func() -> void {
  p == p; // ok
  p == &f; // error (escalates because of f)

  p = &g; // ok
  p = &f; // error (escalates because of f)
}
```

Calling the null reflection value non-consteval-only is really what the consteval-only value model is about.

Also, notably, if the null reflection value is not consteval-only, then the implementation does not need any consteval-only type machinery — which is otherwise still necessary to ensure that all the appropriate initializations escalate. It's just simpler.

§ 3.3 Differences in Treatment Between Consteval-only Types and Consteval-only Values

If we transition to having our model be based on consteval-only *values* rather than consteval-only *types*, mostly the same issues are diagnosed — if for different reasons. But there are a few instances where the consteval-only type rule does diagnose a misuse but the consteval-only value does not.

Since the null reflection is *not* consteval-only (see previous section), code using a default-initialized `std::meta::info` is just allowed:

```
// consteval-only type model: this is ill-formed
// consteval-only value model: this is fine
std::meta::info null;

// both models: this is ill-formed
std::meta::info type = ^^int;
```

Some other examples might be diagnosed differently:

```
// type model: this is ill-formed because it is not declared consteval
// value model (regardless of null treatment): this is fine
auto f(std::meta::info) -> void { }

// null is not consteval-only, so this is valid in the value model
auto g(std::meta::info r) -> std::meta::info { return r; }

constexpr std::meta::info r = ^^int;

// type model: p has consteval-only type so must be declared constexpr
// value model: &r has consteval-only value, so the resulting object must
        be constexpr
std::meta::info const* p = &r;
```

Some other examples might be potentially useful:

```
// consteval-only type: this is ill-formed because v has consteval-only
        type
// consteval-only value: there is no actual reflection here, so this is
        fine
auto v = std::variant<std::meta::info, int>(42);

// ditto
struct C { std::meta::info const* p; };
auto c = C{.p = nullptr};
```

§ 3.4 Ancillary Benefits of Consteval-only Values

The consteval-only value rule gives us a way to keep reflections at compile-time with a simpler, more-easily-enforceable rule than the consteval-only type rule. But the consequences aren't just that some useless code is allowed to hang around. It also provides us significant ancillary benefits.

§ 3.4.1 Variant Visitation

P3603 was motivated by the variant visitation example:

```
#include <array>

template <class T, class U>
```

```
struct Variant {
    union {
        T t;
        U u;
    };
    int index;

    constexpr Variant(T t) : t(t), index(0) { }
    constexpr Variant(U u) : u(u), index(1) { }

    template <int I> requires (I < 2)
    constexpr auto get() const -> auto const& {
        if constexpr (I == 0) return t;
        else if constexpr (I == 1) return u;
    }
};

template <class R, class F, class V0, class V1>
struct binary_vtable_impl {
    template <int I, int J>
    static constexpr auto visit(F&& f, V0 const& v0, V1 const& v1) -> R {
        return f(v0.template get<I>(), v1.template get<J>());
    }

    static constexpr auto get_array() {
        return std::array{
            std::array{
                &visit<0, 0>,
                &visit<0, 1>
            },
            std::array{
                &visit<1, 0>,
                &visit<1, 1>
            }
        };
    }

    // This constexpr variable will be initialized to
    // an array of pointers to consteval functions.
    static constexpr std::array fptrs = get_array();
};

template <class R, class F, class V0, class V1>
constexpr auto visit(F&& f, V0 const& v0, V1 const& v1) -> R {
    using Impl = binary_vtable_impl<R, F, V0, V1>;
    return Impl::fptrs[v0.index][v1.index]((F&&)f, v0, v1);
}

consteval auto func(const Variant<int, long>& v1, const Variant<int,
        long>& v2) {
    return visit<int>([](auto x, auto y) consteval { return x + y; }, v1,
        v2);
}

static_assert(func(Variant<int, long>{42}, Variant<int, long>{1729}) ==
        1771);
```

Note that there are no consteval-only types in this example, but it is representative of the case where the `variant` being visited actually has reflections in it. This example is ill-formed today. The linked paper has a longer description, but basically `fptrs` is a `constexpr` variable that is initialized to an array of pointers to consteval functions. That is disallowed today.

But with the consteval-only value rule, that's fine — it just becomes an immediate variable. That causes usage of it to escalate: the particular specialization of `visit` becomes a `consteval` function, at which point everything just works. This means that variant visitation with existing implementations using variants containing `std::meta::info` or otherwise visiting with a `consteval` function start to work without any code changes required.

§ 3.4.2 Consteval Functions as Constant Template Arguments

Similarly, use of `consteval` functions will explode with C++26 with the adoption of Reflection. And one such use will be as constant template parameters. This example also was noted in P3603R1:

```
consteval auto always_false() -> bool { return false; }

static_assert(std::not_fn(always_false)());   // OK
static_assert(std::not_fn<always_false>()()); // ill-formed
```

Status quo is this is ill-formed (although gcc accepts). With this change, the example becomes well-formed with the desired semantics, with no library code change necessary.

§ 3.4.3 Consteval-only Allocation (Future Work)

Not proposed in this paper, but consider:

```
template for (constexpr std::meta::info m : members_of(type, ctx)) { ...
        }
```

This internally constructs a `constexpr` variable of type `std::vector<std::meta::info>`, which requires allocation (unless `type` happens to have no members — which again, not to belabor the point, but this rule is based on the *value* of the range, not its type). Now, non-transient `constexpr` allocation is a problem that we're trying to solve anyway — but in this case, the values we're talking about are consteval-only values. That allocation *cannot* persist to runtime anyway. And if it cannot persist to runtime, then there's no actual non-transient allocation problem to be solved.

The consteval-only value model gives us a clear path to simply accepting the above formulation, without having to either wait for a general solution or require the user to wrap every call in `define_static_array`. We are not proposing this for C++26 though.

§ 3.4.4 Consteval Variables (Future Work)

Not proposed in this paper, but consider the desire to have *variables* that do not persist to runtime — `consteval` variables. An important benefit of `consteval` variables is that they are *guaranteed* to not occupy

space at runtime. You just don't hit issues like this. In the consteval-only value model, adopting `consteval` variables is trivial since it already fits cleanly into the model:

— allow declaring variables `consteval` in 9.2.6 [dcl.constexpr],

— change the definitions of *potentially-constant* and *usable in constant expressions* to include variables declared `consteval` as well as `constexpr` in 7.7 [expr.const] (and possibly in some other places), and lastly

— extend the definition of *immediate object* (diff against the proposing wording below):

a A value is *consteval-only* if it is either

(a.1) — a reflection value ([basic.fundamental]) that is not the null reflection value or

(a.2) — a pointer or pointer-to-member that points to either an immediate object or an immediate function.

b An object is an *immediate object* if its complete object ~~has either~~

(b.1) — has a constituent value that is consteval-only, ~~or~~

(b.2) — has a constituent reference that refers to either an immediate object or an immediate function~~.~~ , or

(b.3) — is the object declared by a `consteval` variable.

c Every immediate object shall be

(c.1) — the object associated with a constexpr or consteval variable or a subobject thereof,

(c.2) — a template parameter object ([temp.param]) or a subobject thereof, or

(c.3) — an object whose lifetime begins and ends during the evaluation of a manifestly constant-evaluated expression.

That's it.

Given that the type of `i` in `consteval int i = 42;` will be either `int` or `int const` (depending on what spelling we want to pursue for mutability), and very likely not either `consteval int` or `consteval int const`, the consteval-only type model has no solution for this problem and would require either introducing the consteval-only value model anyway or requiring the user to first produce a consteval-only type.

§ 3.5 Implementation Experience

Barry implemented the consteval-only value approach in his fork of the p2996 fork of clang.

This is not exactly the same design as in this paper: it also supports *explicit* immediate variables, so you'll see some test cases that declare `consteval` variables. But otherwise, the implementation has been updated to implicitly escalate `constexpr` variables initialized with an immediate constant expression to `consteval`, so it just ends up being an extension.

§ 4 Proposal

We propose to *replace* the existing consteval-only type model with a consteval-only value model, where a consteval-only value is defined as being:

1. a non-null reflection,
2. a constituent pointer or reference to an immediate function, or
3. a constituent pointer or reference to an immediate object

where an immediate object is an object that has consteval-only value somewhere and and an immediate variable is a `constexpr` variable whose initialization produces a consteval-only value. Note that unlike [P3603R1], this paper does not propose the ability to *explicitly* declare an immediate variable. Explicit `consteval` variables will be a C++29 feature.

All of the relevant rules in 7.7 [expr.const] around escalation will change to consider consteval-only values and immediate variables instead of consteval-only types. The type trait `is_consteval_only` will be removed. The other use of consteval-only type is the mandates on `bit_cast`, which can change to add `std::meta::info` to the category of types with constexpr-unknown representation — [CWG2765] (Address comparisons between potentially non-unique objects during constant evaluation) — since pointers and references are already rejected, there likewise isn't any issue here with incompleteness.

§ 4.1 Wording

Remove the consteval-only type rule in 6.9.1 [basic.types.general]/12:

12 ~~A type is *consteval-only* if it is~~

(12.1) — ~~std::meta::info,~~

(12.2) — ~~ev T, where T is a consteval-only type,~~

(12.3) — ~~a pointer or reference to a consteval-only type,~~

(12.4) — ~~an array of consteval-only type,~~

(12.5) — ~~a function type having a return type or any parameter type that is consteval-only,~~

(12.6) — ~~a class type with any non-static data member having consteval-only type, or~~

(12.7) — ~~a type "pointer to member of class C of type T", where at least one of C or T is a consteval-only type.~~

~~Every object of consteval-only type shall be~~

(12.8) — ~~the object associated with a constexpr variable or a subobject thereof,~~

(12.9) — ~~a template parameter object ([temp.param]) or a subobject thereof, or~~

(12.10) — ~~an object whose lifetime begins and ends during the evaluation of a core constant expression.~~

~~Every function of consteval-only type shall be an immediate function ([expr.const]).~~

Change the [CWG2765] definition of constexpr-unknown representation in 7.7 [expr.const]/x:

x A type has *constexpr-unknown representation* if it

(x.1) — is a union,

(x.2) — is `std::meta::info`,

(x.3) — is a pointer or pointer-to-member type,

(x.4) — is volatile-qualified,

(x.5) — is a class type with a non-static data member of reference type, or

(x.6) — has a base class or a non-static member whose type has constexpr-unknown representation.

Introduce the concepts of consteval-only value, immediate object, and immediate constant expression around 7.7 [expr.const]/21:

a A value is *consteval-only* if it is either

(a.1) — a reflection value ([basic.fundamental]) that is not the null reflection value or

(a.2) — a pointer or pointer-to-member that points to either an immediate function or either to past the end of an immediate object.

b An object is an *immediate object* if its complete object has either

(b.1) — a constituent value that is consteval-only or

(b.2) — a constituent reference that refers to either an immediate object or an immediate function.

c Every immediate object shall be

(c.1) — the object associated with a constexpr variable or a subobject thereof,

(c.2) — a template parameter object ([temp.param]) or a subobject thereof, or

(c.3) — an object whose lifetime begins and ends during the evaluation of a manifestly constant-evaluated expression.

[*Example 1:*

```
consteval int plus1(int x) { return x + 1; }
template <auto V> struct C { };

auto a = plus1; // error: immediate object not associated with constexpr
        variable
constexpr auto b = plus1; // ok
auto c = C<a>(); // ok
```

```
auto d = ^^int; // error: immediate object not associated with constexpr
        variable
auto e = C<^^char>(); // ok
```

— end example]

Letting *V* be a variable that declares or refers to an immediate object *O*, each expression *E* that odr-uses *V* shall be in an immediate function context; letting *D1* be the innermost declaration that contains *E* and *D2* be defining declaration of *V*, a diagnostic is only required if either *D1* or *D2* is reachable from the other.

21 A *constant expression* is either

(21.1) — a glvalue core constant expression ~~*E* for which~~ that refers to an object or function, or

(21.1.1) — ~~*E* refers to a non-immediate function,~~

(21.1.2) — ~~*E* designates an object *o*, and if the complete object of *o* is of consteval-only type then so is *E*,~~

~~[*Example 2:*~~

```
struct Base { };
struct Derived : Base { std::meta::info r; };

consteval const Base& fn(const Derived& derived) { return derived; }

constexpr Derived obj{.r=^^::}; // OK
constexpr const Derived& d = obj; // OK
constexpr const Base& b = fn(obj); // error: not a constant
        expression
  // because Derived is a consteval-only type but Base is not.
```

~~*— end example*]~~

(21.2) — a prvalue core constant expression whose result object ([basic.lval]) satisfies the following constraints:

(21.2.1) — each constituent reference refers to an object or ~~a non-immediate~~ function,

(21.2.2) — no constituent value of scalar type is an indeterminate or erroneous value ([basic.indet]), and

(21.2.3) — no constituent value of pointer type is ~~a pointer to an immediate function or~~ an invalid pointer value ([basic.compound]). ~~, and~~

(21.2.4) — ~~no constituent value of pointer-to-member type designates an immediate function.~~

(21.2.5) — ~~unless the value is of consteval-only type,~~

(21.2.5.1) — ~~no constituent value of pointer-to-member type points to a direct member of a consteval-only class type~~

(21.2.5.2) — ~~no constituent value of pointer type points to or past an object whose complete object is of consteval-only type, and~~

(21.2.5.3) — ~~no constituent reference refers to an object whose complete object is of consteval-only type.~~

d A constant expression is an *immediate constant expression* if it is either

(d.1) — a glvalue that refers to an immediate object or an immediate function, or

(d.2) — a prvalue whose result object is an immediate object.

Lastly, still in 7.7 [expr.const], reword immediate-escalation in terms of consteval-only values:

23 An expression or conversion is in an *immediate function context* if it is potentially evaluated and either:

(23.1) — its innermost enclosing non-block scope is a function parameter scope of an immediate function,

(23.2) — it is a subexpression of a manifestly constant-evaluated expression or conversion, or

(23.3) — its enclosing statement is enclosed ([stmt.pre]) by the *compound-statement* of a consteval if statement ([stmt.if]).

An invocation is an *immediate invocation* if it [is a potentially-evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. An aggregate initialization is an immediate invocation if it evaluates a default member initializer that has a subexpression that is an immediate-escalating expression.

24 A potentially-evaluated expression or conversion is *immediate-escalating* if it is neither initially in an immediate function context nor a subexpression of an immediate invocation, and

(24.1) — it is an *id-expression* or *splice-expression* that either

(24.1.1) — designates an immediate function or an immediate object, or

(24.1.2) — is a prvalue that computes a consteval-only value;

(24.2) — it is an immediate invocation that either

(24.2.1) — is not a constant expression, or

[*Example 3:*

```
consteval int id(int x) { return x; }
template <auto F>
constexpr auto apply_to(int i) { return F(i); }

auto p = &apply_to<id>; // error: immediate function because F(i)
                        // is not a constant expression
```

— end example]

(24.2.2) — is an immediate constant expression; or

[*Example 4:*

```
consteval std::meta::info refl() { return ^^int; }
template <auto F>
constexpr void ex() {
  auto x = F();
}

auto p = &ex<refl>; // error: immediate function because F()
                    // is an immediate constant expression
```

— end example]

(24.3) — ~~it is of consteval-only type ([basic.types.general]).~~ it is a *reflect-expression*.

[*Example 5:*

```
std::meta::info r;
void normal() {
  r == r; // ok
  r == ^^int; // error
}
```

— end example]

25 An *immediate-escalating* function is […]

26 An *immediate function* is a function that is either

(26.1) — declared with the `consteval` specifier, or

(26.2) — ~~an immediate-escalating function whose type is consteval-only ([basic.types.general]), or~~

(26.3) — an immediate-escalating function *F* whose function body contains ~~either~~

(26.3.1) — an immediate-escalating expression ~~or~~

(26.3.2) — ~~a definition of a non-constexpr variable with consteval-only type~~

whose innermost enclosing non-block scope is *F*'s function parameter scope.

Revert the allowance for `virtual` functions in 11.7.3 [class.virtual]/18 added by [CWG3117] [Drafting note: This allowance was never used in `meta::exception`, its `what()` is still declared `constexpr` — that is [LWG4513] which is no longer a defect]:

18 A ~~class with a~~ `consteval` virtual function shall not override ~~that overrides~~ a virtual function that is not `consteval` ~~shall have consteval-only type ([basic.types.general])~~. A `consteval` virtual function shall not be overridden by a virtual function that is not `consteval`.

Remove the consteval-only type trait from 21.3.3 [meta.type.synop]:

```
namespace std {
  // [meta.unary.prop], type properties
  template<class T> struct is_const;
  template<class T> struct is_volatile;
  template<class T> struct is_trivially_copyable;
  template<class T> struct is_standard_layout;
  template<class T> struct is_empty;
  template<class T> struct is_polymorphic;
  template<class T> struct is_abstract;
  template<class T> struct is_final;
  template<class T> struct is_aggregate;
- template<class T> struct is_consteval_only;

  // ...

  // [meta.unary.prop], type properties
  template<class T>
    constexpr bool is_const_v = is_const<T>::value;
  template<class T>
    constexpr bool is_volatile_v = is_volatile<T>::value;
  template<class T>
                    constexpr     bool     is_trivially_copyable_v     =
        is_trivially_copyable<T>::value;
  template<class T>
    constexpr bool is_standard_layout_v = is_standard_layout<T>::value;
  template<class T>
    constexpr bool is_empty_v = is_empty<T>::value;
  template<class T>
    constexpr bool is_polymorphic_v = is_polymorphic<T>::value;
  template<class T>
    constexpr bool is_abstract_v = is_abstract<T>::value;
  template<class T>
    constexpr bool is_final_v = is_final<T>::value;
  template<class T>
    constexpr bool is_aggregate_v = is_aggregate<T>::value;
- template<class T>
-   constexpr bool is_consteval_only_v = is_consteval_only<T>::value;
}
```

And from the 21.3.6.4 [meta.unary.prop] table:

Template	Condition	Preconditions
`template<class T> struct is_const;`	T is const-qualified ([basic.type.qualifier])	
…	…	…
~~`template struct is_consteval_only;`~~	~~T is consteval-only ([basic.types.general])~~	~~remove_all_extents_t<T> shall be a complete type or cv void.~~

And from 21.4.1 [meta.syn]:

```
namespace std::meta {
  // associated with [meta.unary.prop], type properties
  consteval bool is_const_type(info type);
  consteval bool is_volatile_type(info type);
  consteval bool is_trivially_copyable_type(info type);
  consteval bool is_standard_layout_type(info type);
  consteval bool is_empty_type(info type);
  consteval bool is_polymorphic_type(info type);
  consteval bool is_abstract_type(info type);
  consteval bool is_final_type(info type);
  consteval bool is_aggregate_type(info type);
- consteval bool is_consteval_only_type(info type);
  consteval bool is_signed_type(info type);
  consteval bool is_unsigned_type(info type);
  consteval bool is_bounded_array_type(info type);
  consteval bool is_unbounded_array_type(info type);
  consteval bool is_scoped_enum_type(info type);
}
```

Replace the reference to consteval-only type to consteval-only value in 21.4.4 [meta.reflection.exception]/1:

1 Reflection functions throw exceptions of type `meta::exception` to signal an error. ~~`meta::exception` is a consteval-only type.~~

And the same for `access_context` in 21.4.8 [meta.reflection.access.context]/3:

3 The type `access_context` is a structural, ~~consteval-only,~~ non-aggregate type. Values of type `access_context` are consteval-only values.

And the same for `data_member_options` in 21.4.16 [meta.reflection.define.aggregate]/1:

1 The classes `data_member_options` and `data_member_options::`*`name-type`* ~~are consteval-only types ([basic.types.general]), and~~ are not structural types ([temp.param]). Values of both types are consteval-only values.

Remove the Mandates from `std::bit_cast` in 22.11.3 [bit.cast] (this assumes the new post-[CWG2765] wording). This restriction is now handled in [expr.const]:

2 ~~*Mandates*: Neither `To` nor `From` are consteval-only types ([basic.types.general]).~~

3 *Constant When*: `To` and `From` do not have constexpr-unknown representation ([expr.const]).

§ 4.2 Feature-test Macro

Bump `__cpp_consteval` in 15.12 [cpp.predefined]:

```
- __cpp_consteval 202406L
+ __cpp_consteval 20XXXXL
```

§ 5 References

[CWG2765] CWG. 2023-07-14. Address comparisons between potentially non-unique objects during constant evaluation.
https://cplusplus.github.io/CWG/issues/2765.html

[CWG3117] Daniel Katz. 2025-11-06. Overriding by a consteval virtual function.
https://wg21.link/cwg3117

[CWG3150] Jakub Jelinek. 2026-01-19. Incomplete consteval-only class types.
https://cplusplus.github.io/CWG/issues/3150.html

[LWG4513] Jonathan Wakely. meta::exception::what() should be consteval.
https://wg21.link/lwg4513

[P0929R2] Jens Maurer. 2018-06-06. Checking for abstract class types.
https://wg21.link/p0929r2

[P2996R13] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2025-06-18. Reflection for C++26.
https://wg21.link/p2996r13

[P3603R1] Barry Revzin, Peter Dimov. 2025-10-06. Consteval-only Values and Consteval Variables.
https://wg21.link/p3603r1

[P4132R0] Dan Katz and Wyatt Childers. 2026-03-23. Addressing incomplete consteval-only types.
https://isocpp.org/files/papers/D4132R0.html

[P4135R0] Wyatt Childers. 2026-03-23. Consteval-only Types: A design space overview.
https://isocpp.org/files/papers/P4135R0.pdf